Name: Aditya Bhende

Roll No: 28

Class: C-C2

```
import warnings
warnings.filterwarnings('ignore')

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.svm import SVC
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    classification_report, confusion_matrix
)

sns.set_theme(style='whitegrid')
```

1. Load Dataset & Inspect

```
df = pd.read_csv('ai4i2020.csv')
print("Shape:", df.shape)
df.head()
```

```
Shape: (10000, 14)
```

	UDI	Product ID	Type	Air temperature [K]	Process temperature [K]	Rotational speed [rpm]	Torque [Nm]	Tool wear [min]	Ma fa
0	1	M14860	M	298.1	308.6	1551	42.8	0	
1	2	L47181	L	298.2	308.7	1408	46.3	3	
2	3	L47182	L	298.1	308.5	1498	49.4	5	
3	4	L47183	L	298.2	308.6	1433	39.5	7	

Next steps: Generate code with df | New interactive sheet

```
df.info()
print("\nMissing values per column:")
print(df.isnull().sum())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 14 columns):
 #   Column                  Non-Null Count  Dtype  
---  ------                  --------------  -----  
 0   UDI                     10000 non-null  int64  
 1   Product ID              10000 non-null  object 
 2   Type                    10000 non-null  object 
 3   Air temperature [K]     10000 non-null  float64
 4   Process temperature [K] 10000 non-null  float64
 5   Rotational speed [rpm]  10000 non-null  int64  
 6   Torque [Nm]             10000 non-null  float64
 7   Tool wear [min]         10000 non-null  int64  
 8   Machine failure         10000 non-null  int64  
 9   TWF                     10000 non-null  int64  
 10  HDF                     10000 non-null  int64  
 11  PWF                     10000 non-null  int64  
 12  OSF                     10000 non-null  int64  
 13  RNF                     10000 non-null  int64  
dtypes: float64(3), int64(9), object(2)
memory usage: 1.1+ MB

Missing values per column:
UDI                        0
Product ID                 0
Type                       0
Air temperature [K]        0
Process temperature [K]    0
Rotational speed [rpm]     0
Torque [Nm]                0
Tool wear [min]            0
Machine failure            0
TWF                        0
HDF                        0
PWF                        0
OSF                        0
RNF                        0
dtype: int64
```

```
df.describe()
```

	UDI	Air temperature [K]	Process temperature [K]	Rotational speed [rpm]	Torque [Nm]	
count	10000.00000	10000.000000	10000.000000	10000.000000	10000.000000	100
mean	5000.50000	300.004930	310.005560	1538.776100	39.986910	1
std	2886.89568	2.000259	1.483734	179.284096	9.968934	
min	1.00000	295.300000	305.700000	1168.000000	3.800000	
25%	2500.75000	298.300000	308.800000	1423.000000	33.200000	
50%	5000.50000	300.100000	310.100000	1503.000000	40.100000	1
75%	7500.25000	301.500000	311.100000	1612.000000	46.800000	1
max	10000.00000	304.500000	313.800000	2886.000000	76.600000	2

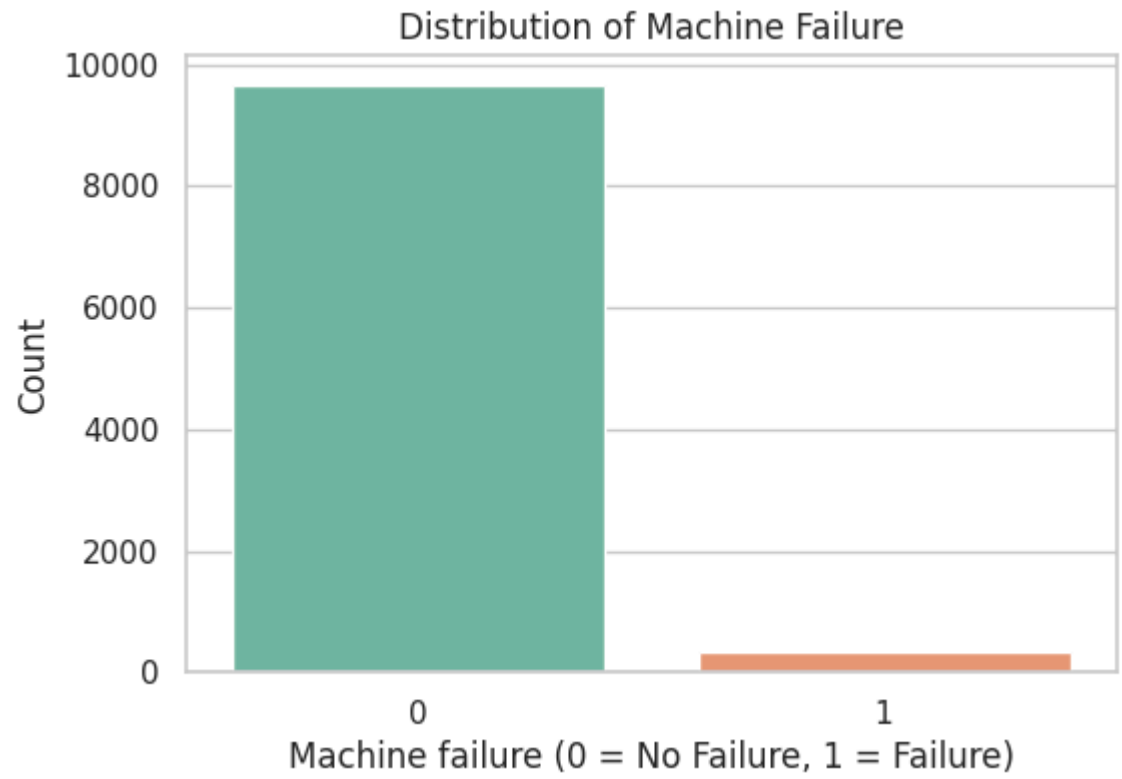
2. Exploratory Data Analysis

Output 1: Distribution of Machine failure (target)

```
counts = df['Machine failure'].value_counts().sort_index()
pct = (counts / len(df) * 100).round(2)

plt.figure(figsize=(6,4))
sns.countplot(x='Machine failure', data=df, palette='Set2')
plt.title('Distribution of Machine Failure')
plt.xlabel('Machine failure (0 = No Failure, 1 = Failure)')
plt.ylabel('Count')
plt.show()

print("Target Class Counts:")
print(f"Class 0 (No Failure) : {counts[0]} samples ({pct[0]}%)")
print(f"Class 1 (Failure)    : {counts[1]} samples ({pct[1]}%)")
```

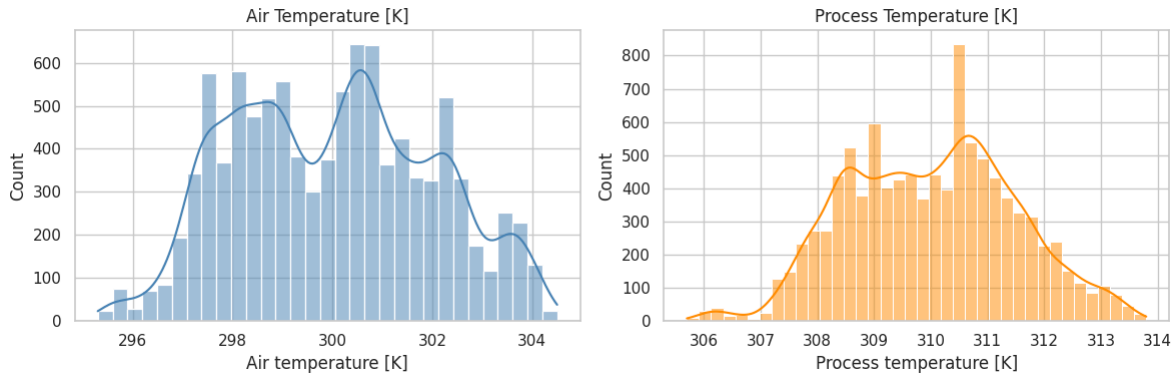



```
Target Class Counts:
Class 0 (No Failure) : 9661 samples (96.61%)
Class 1 (Failure)    : 339 samples (3.39%)
```

Output 2: Temperature distributions

```
fig, axes = plt.subplots(1, 2, figsize=(12,4))
sns.histplot(df['Air temperature [K]'], kde=True, ax=axes[0], color='steelbl
axes[0].set_title('Air Temperature [K]')
sns.histplot(df['Process temperature [K]'], kde=True, ax=axes[1], color='dar
axes[1].set_title('Process Temperature [K]')
plt.tight_layout()
plt.show()

for col in ['Air temperature [K]', 'Process temperature [K]']:
    print(f"{col}: Range [{df[col].min():.1f} - {df[col].max():.1f} K], Mean
```



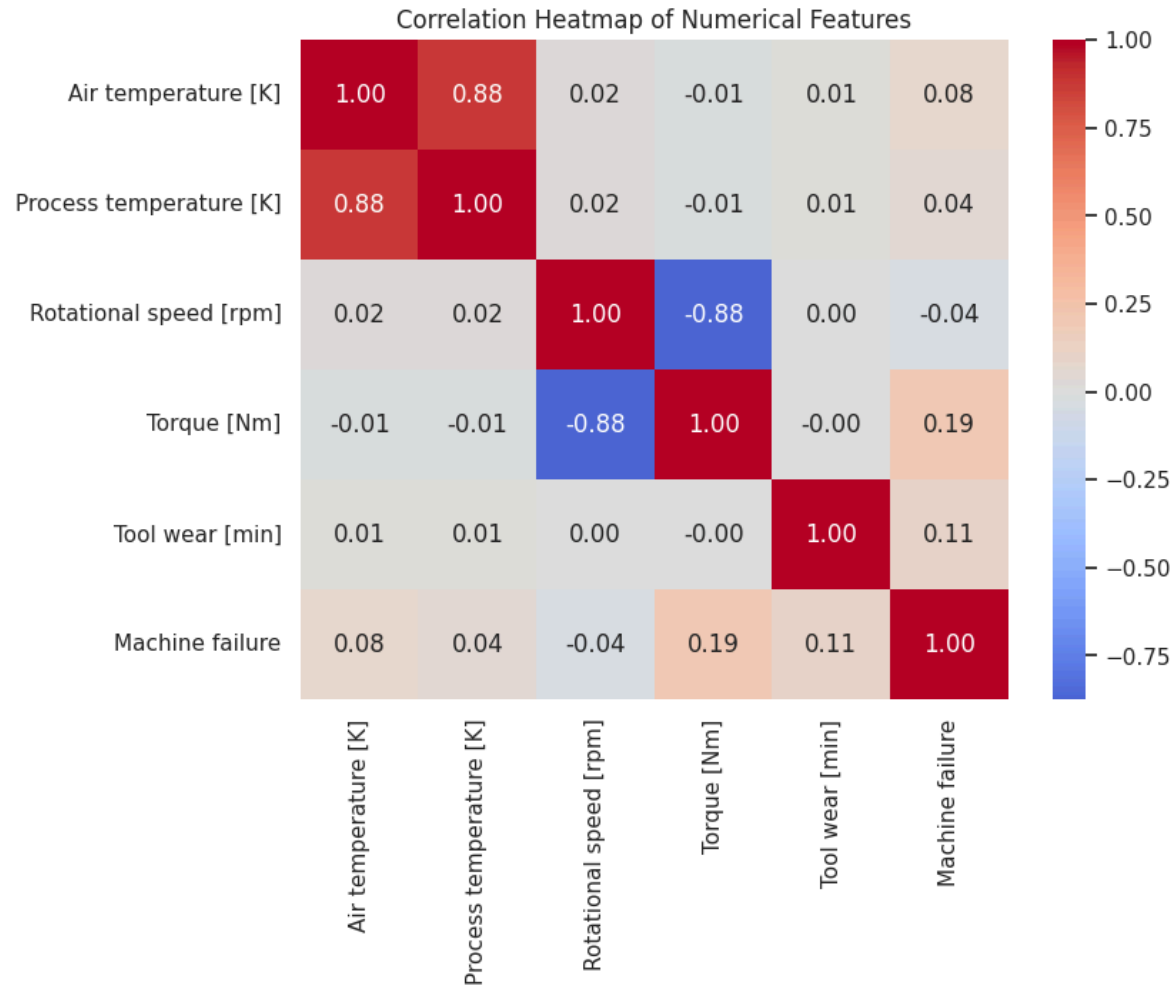

```
Air temperature [K]: Range [295.3 - 304.5 K], Mean: 300.0 K
Process temperature [K]: Range [305.7 - 313.8 K], Mean: 310.0 K
```

Output 3: Correlation heatmap

```
num_cols = ["Air temperature [K]", "Process temperature [K]",
            "Rotational speed [rpm]", "Torque [Nm]", "Tool wear [min]",
            "Machine failure"]
corr = df[num_cols].corr()

plt.figure(figsize=(8,6))
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm', center=0)
plt.title('Correlation Heatmap of Numerical Features')
plt.show()

print("Key Correlation Values:")
print(f"Air Temp vs Process Temp  : {corr.loc['Air temperature [K]','Process
print(f"Rotational Speed vs Torque: {corr.loc['Rotational speed [rpm]','Torq
print(f"Torque vs Machine Failure : {corr.loc['Torque [Nm]','Machine failure
```




```
Key Correlation Values:
Air Temp vs Process Temp  : +0.88
Rotational Speed vs Torque: -0.88
Torque vs Machine Failure : +0.19
```

3. Features, Preprocessing and Train/Test Split

```
X = df[["Type", "Air temperature [K]", "Process temperature [K]",
        "Rotational speed [rpm]", "Torque [Nm]", "Tool wear [min]"]]
y = df["Machine failure"]


categorical_features = ['Type']
numerical_features = [
```

```
    "Air temperature [K]", "Process temperature [K]",
    "Rotational speed [rpm]", "Torque [Nm]", "Tool wear [min]"
]

preprocessing = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), numerical_features),
        ("cat", OneHotEncoder(), categorical_features)
    ]
)


X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
print("Train:", X_train.shape, "| Test:", X_test.shape)
```

```
Train: (8000, 6) | Test: (2000, 6)
```

4. Linear SVM

```
linear_svm = Pipeline(steps=[
    ("Preprocessing", preprocessing),
    ("classifier", SVC(kernel='linear'))
])
linear_svm.fit(X_train, y_train)
y_pred_linear = linear_svm.predict(X_test)
```

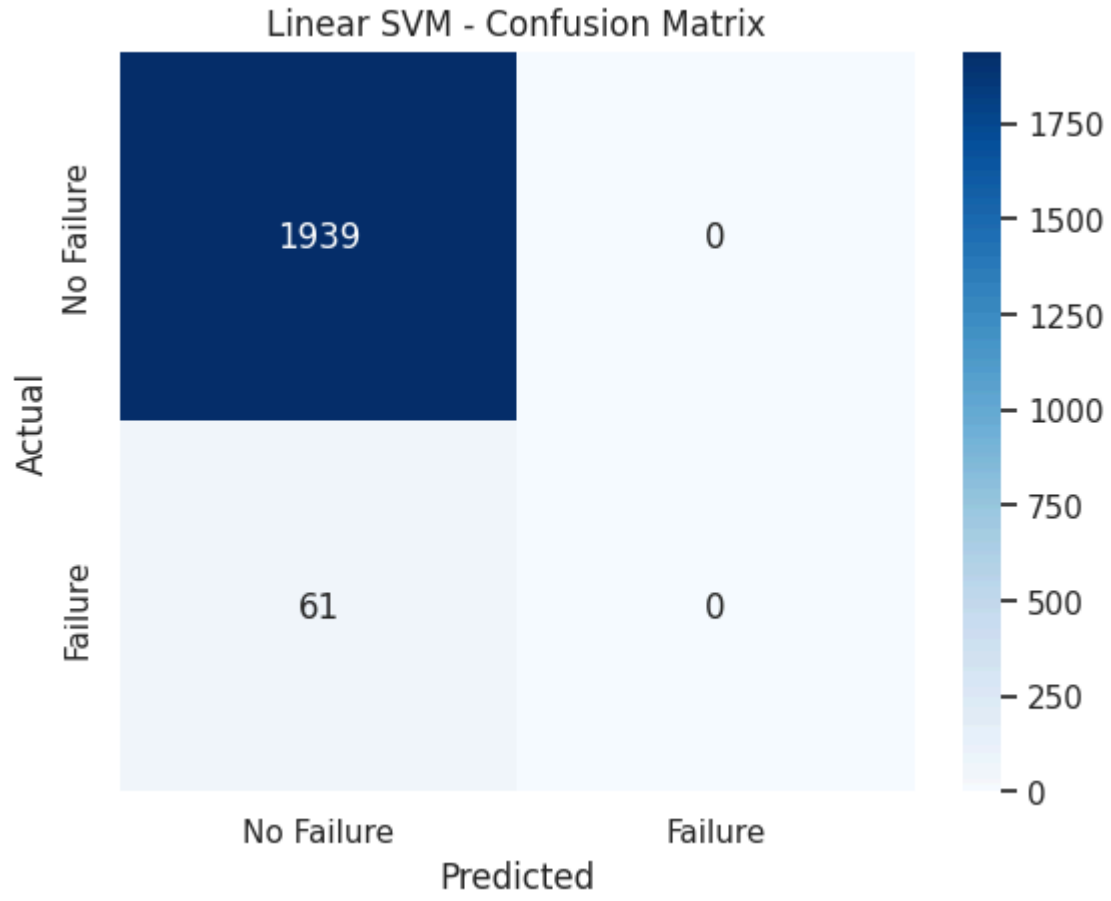
Output 4: Linear SVM results

```
cm_lin = confusion_matrix(y_test, y_pred_linear)
print("Confusion Matrix (Linear SVM):")
print(pd.DataFrame(cm_lin,
      index=['Actual No Failure','Actual Failure'],
      columns=['Predicted No Failure','Predicted Failure']))
print()
print(f"Accuracy: {accuracy_score(y_test, y_pred_linear):.4f} | "
      f"Precision: {precision_score(y_test, y_pred_linear, zero_division=0):
      f"Recall: {recall_score(y_test, y_pred_linear, zero_division=0):.4f} |
      f"F1-Score: {f1_score(y_test, y_pred_linear, zero_division=0):.4f}")

sns.heatmap(cm_lin, annot=True, fmt='d', cmap='Blues',
            xticklabels=['No Failure','Failure'], yticklabels=['No Failure',
plt.title('Linear SVM - Confusion Matrix'); plt.xlabel('Predicted'); plt.yla
plt.show()
```

```
Confusion Matrix (Linear SVM):
                  Predicted No Failure  Predicted Failure
Actual No Failure                 1939                  0
Actual Failure                      61                  0

Accuracy: 0.9695 | Precision: 0.0000 | Recall: 0.0000 | F1-Score: 0.0000
```



5. RBF Kernel SVM

```
rbf_svm = Pipeline(steps=[
    ("Preprocessing", preprocessing),
    ("classifier", SVC(kernel='rbf'))
])
rbf_svm.fit(X_train, y_train)
y_pred_rbf = rbf_svm.predict(X_test)
```

Output 5: RBF SVM results

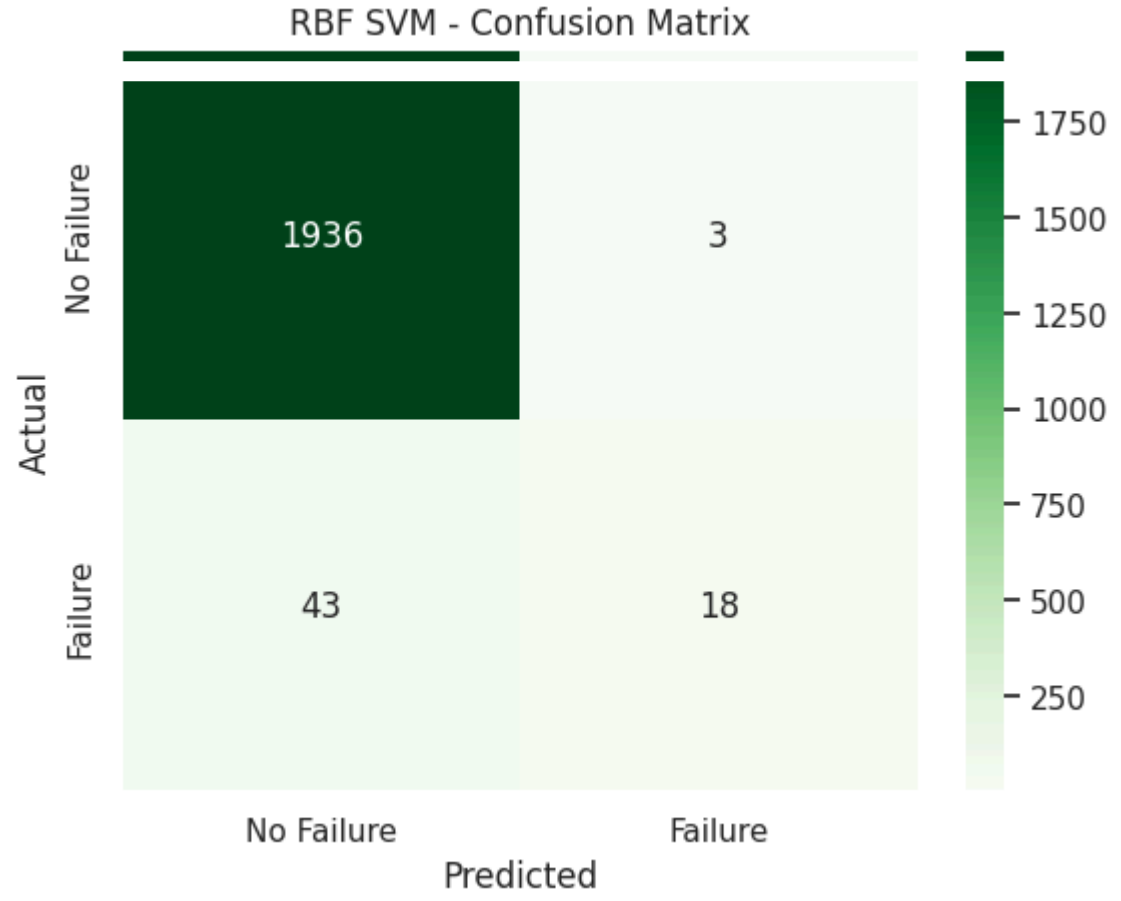
```
cm_rbf = confusion_matrix(y_test, y_pred_rbf)
print("Confusion Matrix (RBF SVM):")
print(pd.DataFrame(cm_rbf,
      index=['Actual No Failure','Actual Failure'],
      columns=['Predicted No Failure','Predicted Failure']))
print()
print(f"Accuracy: {accuracy_score(y_test, y_pred_rbf):.4f} | "
      f"Precision: {precision_score(y_test, y_pred_rbf, zero_division=0):.4f
```

```
      f"Recall: {recall_score(y_test, y_pred_rbf, zero_division=0):.4f} | "
      f"F1-Score: {f1_score(y_test, y_pred_rbf, zero_division=0):.4f}")

sns.heatmap(cm_rbf, annot=True, fmt='d', cmap='Greens',
            xticklabels=['No Failure','Failure'], yticklabels=['No Failure',
plt.title('RBF SVM - Confusion Matrix'); plt.xlabel('Predicted'); plt.ylabel
plt.show()
```

```
Confusion Matrix (RBF SVM):
                  Predicted No Failure  Predicted Failure
Actual No Failure                 1936                  3
Actual Failure                      43                 18

Accuracy: 0.9770 | Precision: 0.8571 | Recall: 0.2951 | F1-Score: 0.4390
```



6. Model Comparison

Output 6: Performance metrics summary

```
comparison = pd.DataFrame({
    'Model': ['Linear SVM', 'RBF SVM'],
    'Accuracy': [accuracy_score(y_test, y_pred_linear), accuracy_score(y_te
    'Precision': [precision_score(y_test, y_pred_linear, zero_division=0),
                  precision_score(y_test, y_pred_rbf, zero_division=0)],
    'Recall': [recall_score(y_test, y_pred_linear, zero_division=0),
               recall_score(y_test, y_pred_rbf, zero_division=0)],
    'F1 Score': [f1_score(y_test, y_pred_linear, zero_division=0),
```

```
                f1_score(y_test, y_pred_rbf, zero_division=0)]
})
comparison_round(4)
```

	Model	Accuracy	Precision	Recall	F1 Score
0	Linear SVM	0.9695	0.0000	0.0000	0.000
1	RBF SVM	0.9770	0.8571	0.2951	0.439

```
print("Classification report - RBF SVM")
print(classification_report(y_test, y_pred_rbf, zero_division=0))
```

```
Classification report - RBF SVM
              precision    recall  f1-score   support

           0       0.98      1.00      0.99      1939
           1       0.86      0.30      0.44        61

    accuracy                           0.98      2000
   macro avg       0.92      0.65      0.71      2000
weighted avg       0.97      0.98      0.97      2000
```

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.